Selecting Reliable LLM Reasoning

DASE7506 Mini-Project 2 | TabM with Adaptive Majority-of-the-Bests 20 October 2026 | mp2-tabm-mob-c914bf67099a

1. Research question and outcome

Can candidate comparisons and answer-level aggregation improve a small MLP selector while remaining within the course's CPU and asset limits? The hypothesis is that a candidate's relative representation and agreement with other candidates provide information beyond its individual embedding.

The retained system combines three randomly initialized TabM models, each with eight shared-parameter branches, and adaptive Majority-of-the-Bests (MoB). It achieved the highest validation accuracy among the recorded configurations: 749/798 (93.86%). Its official public-test result is 382/467 (81.80%). Inference assets total 3.5510 MiB. The recorded frozen-model reproduction took 0.3183 seconds for complete local test prediction plus scoring.

2. Task and experimental setup

Each question has four candidate solutions and fixed 1024-dimensional representations. The selector returns one candidate index from 0 to 3. The unchanged official scorer checks only the final numeric answer; any correct candidate earns one point. Several candidates may be correct.

The supplied split sizes are 4,755 training, 798 validation and 467 test questions. Only training inputs and references fit parameters or normalization statistics. Validation determines early stopping and method selection. Training keeps all four candidates of each question together. No external dataset, extra pretrained weights, question-ID lookup or generation API is used.

3. Candidate and group representations

For candidate j, the input concatenates its supplied embedding e_j, the difference from the four-candidate mean, and 30 label-free metadata features:

```
x_j = concat(e_j, e_j - mean(e_1, ..., e_4), metadata_j)
dim(x_j) = 1024 + 1024 + 30 = 2078
```

Metadata includes answer parseability, answer-group counts, numeric answer properties, text length, question-number overlap, formatting cues, simple arithmetic-check statistics and relative length. All features are computed from the provided inputs. Mean and standard deviation are fitted over training candidates only; the standard-deviation floor is 1e-6.

Arithmetic checks use a restricted AST evaluator for numeric constants and addition, subtraction, multiplication and division. They are weak features, not proofs of valid reasoning. The official numeric parser is unchanged.

4. Architecture, objective and training

Each TabM has two hidden layers of width 128. Eight branches share the main weight matrices but use branch-specific input/output scales, biases and final heads. Three independent seeds produce 24 branch predictions in total. Each seed has 305,144 trainable parameters.

Let y_j indicate whether the official parser maps candidate j to the reference answer, and let z_j be a branch's logit. Each branch minimizes:

```
L = BCEWithLogits(z, y)
+ log(sum_j exp(z_j)) - log(sum_{j: y_j=1} exp(z_j))
```

The first term learns candidate correctness. The second encourages mass on any correct candidate without treating other correct answers as negatives. It is zero for an all-correct group. Losses are averaged across branches before backpropagation.

Setting	Value
Seeds / branches per seed	42, 123, 2026 / 8
Hidden layers / dropout	128, 128 / 0.15
Optimizer / learning rate / weight decay	AdamW / 0.002 / 0.0003
Batch size / maximum epochs / patience	128 questions / 80 / 12
Gradient clipping / training device	Norm 5 / CPU, four numerical threads
Selected epochs / completed epochs	3, 12, 8 / 15, 24, 20

Early stopping uses validation selection accuracy from mean sigmoid scores, with lower validation loss breaking ties. The final aggregation setting is selected separately from the recorded validation comparison. Final inference averages raw logits across all 24 branches.

5. Adaptive answer aggregation

The original MoB implementation groups equivalent answers using the official parser. Among n parseable candidates, it sorts verifier scores. For ascending rank r, the probability of being the best candidate in m uniform draws with replacement is:

```
p_r(m) = ((r + 1) / n)^m - (r / n)^m, r = 0, ..., n-1
```

Probabilities are summed by final answer. Adaptive MoB uses the original q=0.75 rule to choose m from candidate resampling sizes using changes between adjacent answer distributions. The selected answer maps to an existing candidate. Unparseable candidates are excluded; if all are unparseable, the highest-scoring candidate is returned as a defined fallback.

6. Validation evidence and ablation

Comparisons use the same official validation bank and numeric scoring protocol. The baseline and feature-ablation implementations are maintained in experiments/; one fixed-setting replay reproduced every row below.

Method	Correct / 798	Accuracy
Answer-majority baseline	688	86.22%
Initial embedding MLP	721	90.35%
Initial MLP + parseability mask	725	90.85%
Feature MLP ensemble, no relative embedding	741	92.86%
Feature MLP ensemble, relative embedding	746	93.48%
TabM + adaptive MoB (selected)	749	93.86%

Adding relative embeddings to the feature MLP increased the score by five questions, from 741 to 746. This comparison supports candidate-relative information, but it does not isolate each metadata feature's contribution. The total increase from the initial MLP to the final model is 28 questions (3.51 percentage points).

The key aggregation ablation holds all final TabM weights fixed. The seven rows below are reproduced by scripts/train.py --ablate-checkpoint and use validation references only.

Branch aggregation	Answer selection	Correct	Accuracy
Mean logits	Highest score	748	93.73%
Mean sigmoid	Highest score	745	93.36%
Mean softmax	Highest score	748	93.73%
Mean sigmoid	Answer-group sum	733	91.85%
Mean softmax	Answer-group sum	743	93.11%
Mean logits	Fixed-size MoB	720	90.23%
Mean logits	Adaptive MoB	749	93.86%

Adaptive MoB adds only one correct answer over plain mean-logit selection (0.13 percentage points). This is a small observed gain, not evidence of statistical superiority. Simple answer-probability summation and fixed-size MoB performed worse. Correlated incorrect candidates can make consensus misleading; learned scores and aggregation must be evaluated together.

Answer-group summation was inspired by LEVER's execution-result aggregation. This task lacks executable programs and generation probabilities, so the ablation is a restricted adaptation rather than a reproduction of LEVER.

7. Final result and resource use

Measurement	Result
Validation accuracy	749/798 = 93.859649%
Public-test accuracy	382/467 = 81.798715%
Uncompressed inference assets	3,723,503 bytes = 3.5510 MiB
Complete local test evaluation	0.3183 seconds
Hardware / runtime	Apple M5, 16 GB; Python 3.12.13; NumPy 2.5.3
Limits / reference measurement	16 MiB and 30 seconds; Xeon not measured

The asset count includes all three members, normalization arrays and configuration files. Inference uses NumPy with exported weights and does not import Torch or TabM. Timing includes startup, imports, data and asset loading, feature extraction, all ensemble members and the independent scoring process. The two processes run sequentially with numerical-library thread limits of four. The reported time is the recorded complete evaluation measurement.

The selected TabM training/comparison stage took 26.86 seconds. The initial MLP and feature-MLP stages took 3.12 and 8.87 seconds, respectively. Across the recorded neural search, 62 training runs took about 520.03 seconds of program runtime. This includes rejected ideas; it excludes setup, coding, tests and exploratory diagnostics. Separate verification used 27.17 seconds for three final-model runs and 12.31 seconds for the seven-run baseline comparison replay, including validation. These replays are excluded from the original search cost.

The initial MLP used 0.5262 MiB and the feature MLP 3.1491 MiB of assets. The final model remains well below the asset limit, but its small gain over the feature MLP should be weighed against extra parameters and training cost. Reference-machine timing eligibility is not established by local measurements.

8. Failure analysis

Of 798 validation questions, 355 contain at least two distinct parseable answers. The frozen model gets 306/355 (86.20%) on this disagreement subset and 443/443 on the remaining groups. Thus the full validation accuracy is inflated by easier agreement groups. The test bank requires answer disagreement, so the full validation and test percentages are not like-for-like difficulty measures.

One validation failure asks for nap hours over 70 days, given three two-hour naps per week. The model selects 420 rather than 60. The chosen trace computes 3 x 2 = 6 and 6 x 70 = 420 correctly, but omits conversion from days to weeks. Correct local arithmetic does not establish correct units or interpretation.

Another failure starts with four pies, sells one and gives one away. The chosen trace incorrectly retains three pies, then performs consistent downstream arithmetic and returns six pieces instead of four. These examples show why text-level arithmetic features cannot verify a whole reasoning chain.

9. Limitations and reproducibility

The author reports that models were selected using validation performance. Test scores were inspected after model selection for comparisons and, according to the author, did not determine model selection. Details are documented in docs/ai_disclosure.md.

The validation bank was reused across several development rounds, so small differences may reflect selection noise. The adaptive-MoB improvement is one question and has not been confirmed on an independent audit bank. No generalization claim is made beyond the measured course banks.

Formatting and length features may exploit superficial correlations. No individual-feature ablation establishes that arithmetic checking causes the overall improvement. The official metric grades final answers, not intermediate reasoning validity. The two inspected failures are illustrative, not a complete taxonomy of errors.

The package includes one final inference bundle. Local fixed-setting reproduction recovered the final model and all seven baseline runs with byte-identical checkpoint files. Seven focused tests passed, including export parity and supervision restrictions. Check details are retained in results/verification.json and results/reproduction/.

README.md and docs/reproduction.md provide installation, training, ablation, evaluation and audit-input commands. Pinned training dependencies include Torch 2.8.0, TabM 0.0.3 and rtdl-num-embeddings 0.0.12. results/release.json identifies the source version and matching checkpoint hashes; results/evaluation contains the official JSON results and complete process measurements. experiments/README.md documents the baseline replay. Comparison summaries with clarified selection metadata are in results/comparisons/; the unmodified experiment JSON archive and hash manifest are in results/history/. Download the supplied data from the exact course revision specified in docs/reproduction.md.

10. AI assistance and reuse

OpenAI Codex helped me complete this project.

TabM was used with random initialization. The original MoB algorithm is vendored without edits under its MIT license. LEVER is a methodological reference only. The original course data, parser and scorer remain unchanged. The retained GSM8K license and notice describe data attribution.

References

[1] DASE7506 MP2 course guide and code. https://github.com/xudongwu-0/DASE7506-MP2 (revision bb801f4f2e7629d270df765ab12c333b2dc616bb).

[2] TabM: Advancing Tabular Deep Learning with Parameter-Efficient Ensembling. https://arxiv.org/abs/2410.24210 ; https://github.com/yandex-research/tabm . Package 0.0.3, Apache-2.0.

[3] Majority-of-the-Bests. https://github.com/arakhsha/mob ; https://openreview.net/forum?id=aEAbRPXV37 . Revision 7925cdd66c409f03c47d7bc02cd8121ebab5dc0d, MIT.

[4] LEVER: Learning to Verify Language-to-Code Generation with Execution. https://arxiv.org/abs/2302.08468 ; https://github.com/niansong1996/lever . Inspected revision 8173b747e6e1351176fe8ae6e3885db1bc63ab04.